

Benchmarking Nano YOLO Detectors for Fine-Grained Military Aircraft Detection on MAR20

Ertugrul DOGRULUK¹, Durdu Mehmet KÖSE¹, Pedro Pimenta^{2,3}, Ibrahim ARUK¹, Hakan AÇIKGÖZ¹

¹Artificial Intelligence and Intelligent Systems, Istiklal Science and Technology University, Kahramanmaraş, Türkiye

²LIACC – Artificial Intelligence and Computer Science Laboratory, Portugal

³N2i – Núcleo de Investigação do Instituto Politécnico da Maia, Portugal

ertugrul.dogruluk@istiklal.edu.tr, 25050011032@ogr.istiklal.edu.tr, ppimenta@umaia.pt,

ibrahim.aruk@istiklal.edu.tr, hakan.acikgoz@istiklal.edu.tr

Abstract—Recognizing military aircraft in remote sensing images means finding small objects on cluttered aprons and telling apart airframes that look almost identical from above. This study compares five nano detectors, YOLOv5nu, YOLOv8n, YOLO11n, YOLO12n and YOLO26n, on the MAR20 benchmark under one shared training configuration. The 3,842 images were re-partitioned into class-stratified training, validation and test subsets of 2,691, 770 and 381 images, and the test subset was used only once, after checkpoint selection. A perceptual-hash audit found a near-duplicate training image for 0.5% of the test images. YOLO12n obtained the highest test scores, 97.29% mAP@0.5 and 80.45% mAP@0.5:0.95, yet all five models fell within 0.33 and 0.67 percentage points on these two measures, and YOLO12n needed the longest training time. The E-8 and F-15 classes remained the hardest for every model. A confusion matrix shows that almost all remaining errors are wrong type labels rather than missed aircraft. We also compiled the numeric MAR20 results of ten published studies and compared them with our models, recording the box type, data split and metric behind each figure. Most published results rely on the official split or on oriented boxes, so our higher scores are best read as a transparent baseline for the re-partitioned protocol rather than as a new state of the art.

Keywords—MAR20, military aircraft detection, fine-grained object detection, remote sensing, YOLO, benchmark.

I. INTRODUCTION

Satellite and aerial imagery let analysts follow activity at airfields on a large scale, but reading the type of an aircraft from above is harder than it sounds. In MAR20 [1], many aircraft cover only a few dozen pixels, they stand in dense rows on busy aprons, and several types share the same basic layout. The E-3 and the E-8, for instance, are both derived from the same four-engine airliner, and from a nadir view the E-3 rotodome is often the only obvious difference. A detector therefore has to do two things well at once: find every aircraft and name its type.

MAR20 has become the common benchmark for this problem. It holds 3,842 images from 60 military airfields with 22,341 aircraft of 20 types, annotated with both horizontal and oriented boxes. Work on it ranges from modified and re-evaluated YOLO detectors [2]–[5] to oriented and fine-grained detectors [6]–[11], and even to few-shot and open-set settings [12], [13]. The published numbers rarely line up, however. Some studies keep the official split and others re-divide the data; some score horizontal boxes and others oriented ones; some report only mAP@0.5. Anyone who simply wants to know how a small, off-the-shelf detector performs on MAR20 has to untangle all of this first.

Our aim is practical. We trained the five nano detectors currently shipped with the Ultralytics library [14] under one configuration, evaluated each of them once on a held-out test subset, and then set the results beside the literature with the protocol differences written out.

The contributions are threefold: 1) a controlled benchmark of YOLOv5nu, YOLOv8n, YOLO11n, YOLO12n and YOLO26n with identical data, schedule and augmentation, reported on a test subset unused in training and checkpoint selection; 2) a consolidated table of published MAR20 results annotated with box type, split and metric, with an explicit statement of which comparisons are meaningful; and 3) a measured near-duplicate audit of our partition and a class-level error analysis linked to difficulties already reported for MAR20.

II. RELATED WORK ON MAR20

A. Horizontal-box detectors

DET-YOLO [2] rebuilds YOLOv8n for military aircraft with C2f_DEF blocks, pruning, a DAT attention module and a retuned loss. On a re-split of 2,765 training, 308 validation and 769 test images it reports 94.7% mAP@0.5 after 80 epochs, against 91.3% for its YOLOv8n baseline and 94.0% for YOLOv5n, with only 1.41 M parameters. Because it also uses a re-split and nano baselines, it is the study closest to ours.

FGA-YOLO [3] adds triple feature fusion, a gather-and-distribute neck and an EMA Slide Loss to a YOLOv8s base. On the official split it reaches 91.3% mAP@0.5 and 69.6% mAP@0.5:0.95, up from 89.0% and 68.3% for YOLOv8s; its weakest types are A15 (F-22), A5 (F-16) and A13 (F-15).

Wang et al. [4] add frequency-aware input enhancement, dual-scale neck alignment and a refined P3 branch to YOLOv5, YOLOv8, YOLO11 and YOLO26 at three scales and three input sizes. At 640 pixels the nano baselines reach 57.1% (YOLO26n) to 70.2% (YOLOv8n) mAP@0.5, and the full configuration lifts YOLO11n to 75.4%. Training ran for 100 epochs; the split is not stated.

Zhao and Tian [5] ablate a P2 head, CBAM attention and WIoU loss on YOLOv8s with horizontal boxes. They keep the official test list, hold out 133 of the 1,331 training images for validation and train for 150 epochs at 800 pixels. Plain YOLOv8s reaches 84.67% mAP@0.5 and 63.33% mAP@0.5:0.95, and CBAM with WIoU raises this to 84.80% and 63.51%. More useful for us is the gap between their subsets: the same model scores 80.16% mAP@0.5:0.95 on the held-out training images but 63.33% on the official test list, and the P2 head that helps on validation does not help on test.

B. Oriented and fine-grained detectors

Several strong results come from oriented detectors built with MMRotate and trained for 36 epochs on the official 1,331/2,511 split. MFCANet [6] extends RTMDet with context-aggregation modules and reaches 85.96% mAP@0.5, 2.13 points above its baseline. PEDNet [7] was designed for ships but tested on MAR20 for generalization, obtaining 80.84% mAP@0.5 and 55.54% mAP@0.5:0.95. Shi et al. [8] inject cross-shaped geometric priors into Oriented RepPoints and report 86.71% mAP@0.5 under the VOC 2007 metric, a gain of 5.42 points; they also note that A7 (E-3) and A11 (E-8) are easily confused. LFDet [9] brings language priors and frequency-domain processing into a one-stage detector and reaches 87.62% mAP@0.5 with a CSPNeXt backbone.

C2IDet [10], a one-stage anchor-free detector with a region-to-region context distributor and a class-aware decoupled focal loss, reaches 84.16% mAP@0.5 with 29.97 M parameters after 120 epochs, and reports a strong two-stage baseline mistaking A11 (E-8) for A7, A17 and A18. SOOD [11] adapts anchors, angle encoding and sample assignment to the rotated shape of each object and reports 85.77% mAP@0.5 (VOC 2007) with 38.99 M parameters; its angle encoder alone lifts A11 by 9.63 points.

MAR20 has also served as a testbed for few-shot detection [12] and open-set detection [13]. These studies answer different questions, so we leave them out of the numeric comparison.

Read together, these studies show that the strongest reports use oriented boxes and the official split, that YOLO-based work usually reports gains over baselines trained under its own settings, and that the baseline for one nano model varies widely, from 70.2% [4] to 91.3% [2] mAP@0.5 for YOLOv8n. This spread is the gap our same-settings comparison tries to narrow.

III. MATERIALS AND METHODS

A. Dataset and partitioning

MAR20 [1] images come from Google Earth and measure about 800×800 pixels; the 20 types, labeled A1–A20, include the SU-35, C-130, C-17, F-16, E-3, E-8, F-15, F-22, KC-135 and KC-10. We used the horizontal-box annotations. A few XML files store an image size of 0×0 , so the true size was read from the image; after this correction all 22,341 boxes were converted.

The official lists assign 1,331 images to training and 2,511 to testing. We instead pooled the 3,842 images and split them 7:2:1 with seed 0, stratifying by the most frequent class in each image so that rare types appear in every subset. Table I shows the outcome. The pooled split has a direct consequence: our training set is about twice as large as the official one, and our test set mixes 256 official test images with 125 official training images. Scores obtained this way cannot be placed on the official leaderboard, a point we return to in Section IV-C.

TABLE I. PARTITION OF MAR20 USED IN THIS STUDY

Subset	Images	Boxes	From official train	From official test
Training	2,691	15,629	932	1,759
Validation	770	4,471	274	496
Test	381	2,241	125	256
Total	3,842	22,341	1,331	2,511

MAR20 crops come from only 60 airfields, so a re-split can place near-identical crops on both sides of the train–test boundary. We checked this with a 64-bit difference hash

(dHash). No image pair across subsets had identical hashes. At a Hamming distance of five bits or less, 2 of the 381 test images (0.5%) had a near-duplicate in training, as did 4 of the 770 validation images (0.5%), and 1 test image (0.3%) matched a validation image. At a looser ten-bit threshold these shares rose to 3.9%, 8.2% and 4.2%. The audit only catches whole-image duplicates. Different crops of the same apron can still share aircraft, so some scene-level overlap cannot be ruled out.

B. Detectors

All five models are the nano checkpoints of Ultralytics 8.4.14 [14]. YOLOv5nu is the anchor-free re-release of YOLOv5n with a split head, YOLOv8n uses C2f blocks and a decoupled anchor-free head, YOLO11n adds C3k2 blocks and a C2PSA attention stage, YOLO12n is built around area attention and R-ELAN blocks, and YOLO26n drops Distribution Focal Loss for NMS-free end-to-end prediction (its checkpoint records end2end=True). With a 20-class head the models hold 2.512, 3.015, 2.594, 2.572 and 2.512 M parameters, counted from the stored checkpoints.

C. Training setup

A single script trained the five models in sequence with the settings in Table II. Every run used the full 200-epoch budget, and the 100-epoch patience was never triggered. The optimizer was left on auto. The logged learning rate peaks at 4.12×10^{-4} after warm-up, which matches the AdamW rate that the auto rule assigns for 20 classes (4.17×10^{-4} before decay), so the nominal lr0 of 0.01 in the argument files was not the rate in effect. All five checkpoints store Ultralytics 8.4.14. The runs were made on the workstation whose Ultralytics logs list an NVIDIA GeForce RTX 4060 with 8 GB of memory, Python 3.12.9 and PyTorch 2.7.1+cu118; the console log of the benchmark script itself was not retained. **Post-training validation and the additional test evaluations were run on a machine with an Apple M4 Max processor with 14 CPU cores and a 32-core GPU, 36 GB of unified memory and a 512 GB solid-state drive. The training, evaluation and audit code is available at <https://github.com/ertugd/mar20v1>.**

TABLE II. SHARED TRAINING CONFIGURATION

Setting	Value
Input size / batch	640 px / 16
Epoch budget / patience	200 / 100
Seed / deterministic mode	0 / enabled
Optimizer / logged peak LR	auto / 4.12×10^{-4}
Final LR factor / weight decay	0.01 / 0.0005
Warm-up / mixed precision	3 epochs / enabled
HSV gains (h, s, v)	0.015, 0.7, 0.4
Translate / scale / horizontal flip	0.1 / 0.5 / 0.5
Mosaic / closed for	1.0 / last 10 epochs
MixUp / CutMix / rotation	off / off / 0°

D. Checkpoint selection and evaluation

For each model we kept best.pt, the epoch with the highest validation fitness, which in Ultralytics 8.4.14 equals mAP@0.5:0.95; the selected epochs were 199, 200, 178, 165 and 190. Each checkpoint was evaluated once on the test subset at 640 pixels with the default candidate confidence threshold of 0.001, an NMS IoU of 0.7 and at most 300 detections per image. Precision and recall are reported at the confidence that maximizes the mean F1 score. Every run used one seed, so we give no confidence intervals.

IV. RESULTS

A. Test performance of the five detectors

TABLE III. TEST RESULTS ON MAR20 (381 IMAGES, 2,241 BOXES)

Model	Params (M)	P (%)	R (%)	mAP@0.5 (%)	mAP@0.5:0.95 (%)	Train (min)
YOLOv5nu	2.512	95.17	94.51	97.19	79.78	74.48
YOLOv8n	3.015	96.04	93.63	97.21	79.77	77.07
YOLO11n	2.594	96.24	94.36	97.06	80.04	79.66
YOLO12n	2.572	95.50	95.00	97.29	80.45	110.71
YOLO26n	2.512	95.42	93.60	96.96	80.28	98.97

YOLO12n leads on both mAP measures, with 97.29% mAP@0.5 and 80.45% mAP@0.5:0.95 (Table III). The margins are thin, though. On mAP@0.5:0.95 it is 0.17 points ahead of YOLO26n, 0.41 ahead of YOLO11n and 0.67 ahead of YOLOv5nu and YOLOv8n, and on mAP@0.5 the whole field spans only 0.33 points. With one seed and 381 test images, gaps of this size could easily reverse in another run, so we read the table as a near tie among five capable models rather than a firm ranking.

At the operating point, YOLO11n has the highest precision (96.24%) and YOLO12n the highest recall (95.00%); the implied F1 scores lie between 94.50% (YOLO26n) and 95.29% (YOLO11n), and YOLOv8n shows the widest precision–recall gap, 2.41 points.

Training cost separates the models more clearly. YOLO12n needed 110.71 min for 200 epochs (about 33 s per epoch), YOLO26n 98.97 min, and the other three 22–24 s per epoch. These times include the per-epoch validation pass on one consumer GPU and say nothing about deployment

latency. YOLOv8n carries the most parameters without a matching gain, while the two smallest models, YOLOv5nu and YOLO26n, stay within 0.67 points of the best mAP@0.5:0.95.

At the selected epochs the validation scores were 98.33–98.49% mAP@0.5 and 80.57–81.33% mAP@0.5:0.95, that is, 0.8–1.4 points above the test values. A small, consistent drop of this kind is what one expects when checkpoints are chosen on validation data, and it suggests that the test subset is not easier than the validation subset.

B. Class-level behavior and training dynamics

For YOLO12n, 18 of the 20 classes exceed 95% AP@0.5. The two exceptions are the same for every model: A11 (E-8) and A13 (F-15). A11 has the lowest AP@0.5 for four of the models, between 83.9% for YOLO12n and 87.6% for YOLO11n; YOLOv5nu is the exception at 92.4%. A13 has the lowest AP@0.5:0.95 for all five models, between 67.2% and 68.8%. In the precision–recall curves of Fig. 1(d), A11 precision drops to about 0.89 at a recall of only 0.25 and falls again beyond 0.8, which points to confident confusion with another type rather than to missed aircraft.

Both classes have been flagged before: Shi et al. [8] and Xu et al. [10] report A11 confusions, and FGA-YOLO [3] finds A13 among its three weakest types. F-15s are also parked in long rows and cover very few pixels (Fig. 3, rightmost scene in the third row of each panel), which makes tight localization at high IoU difficult and explains why A13 is weak on AP@0.5:0.95. By contrast, A15 (F-22), which scored 56.4% in [3], 67.8% in [8] and 80.1% in [11], reaches 95.4% AP@0.5 here, most likely because our partition offers far more training data.

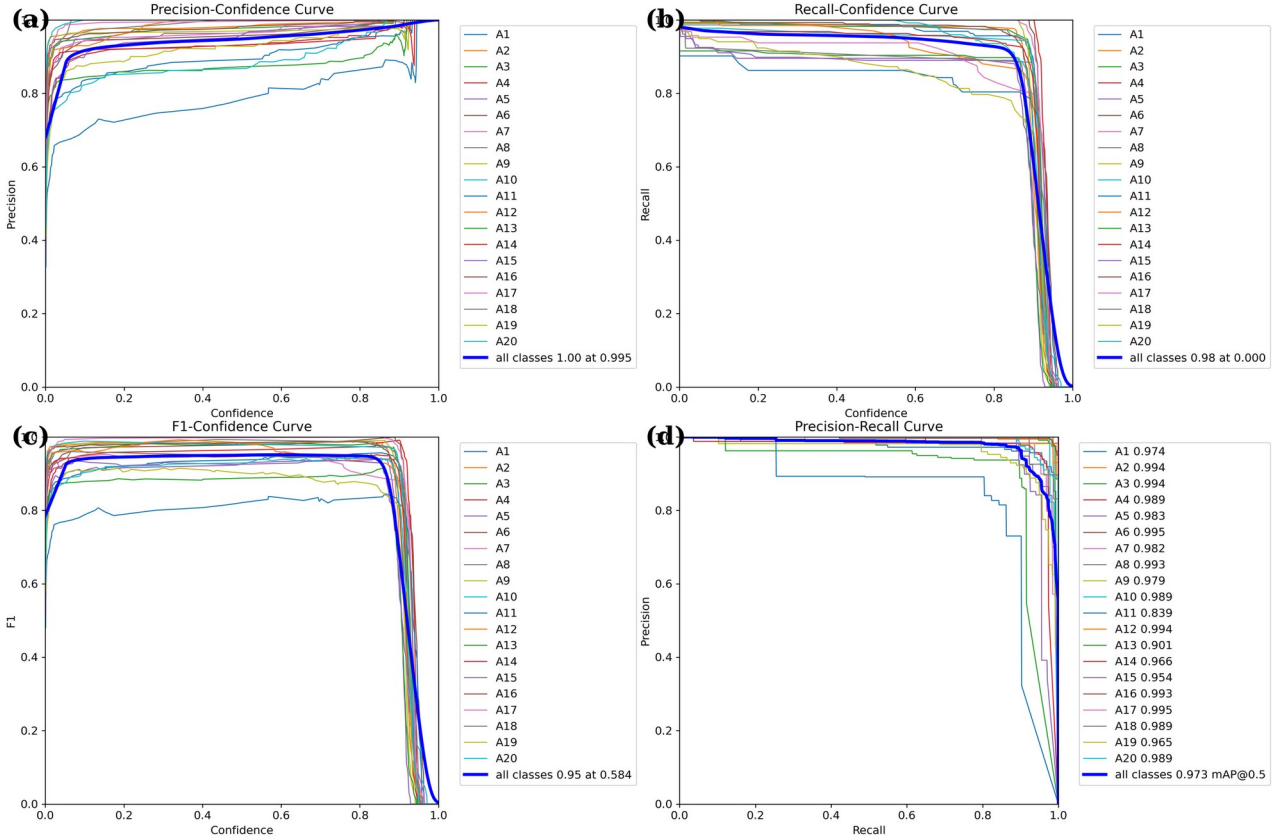


Fig. 1. Test-set curves of YOLO12n: (a) precision–confidence, (b) recall–confidence, (c) F1–confidence and (d) precision–recall.

The per-class tables of the literature tell a consistent story. At IoU 0.5, the three weakest classes are A15, A5 and A13 for FGA-YOLO [3], A15, A18 and A5 for Shi et al. [8], A15, A13

and A20 for LFDet [9], and A13, A5 and A4 for SOOD [11], so small fighters dominate the hard cases. Under the stricter AP@0.5:0.95, YOLO12n shows the same pattern: after A13

(68.8%) and A11 (70.5%), its weakest classes are A15 (73.6%) and A5 (74.0%). At IoU 0.5, however, both fighters exceed 95% in our runs, which suggests that the re-split mainly eases their recognition while tight localization remains hard.

Fig. 2(a) gives the YOLO12n test confusion matrix, with true classes in the columns and predicted classes in the rows. Of the 2,241 test aircraft, 2,146 (95.8%) receive the correct type, 92 (4.1%) are detected but given the wrong type, and only 3 are missed altogether: two B-1Bs and one TU-22M. The detector also places 73 boxes on background, most often labeled SU-34 (11) or SU-35, F/A-18 and SU-24 (9 each). What remains is therefore mainly a naming problem, not a detection problem.

The largest confusions are SU-34 predicted as SU-35 (16 of 118), F-15 as SU-24 (13 of 166), F-16 as F-15 (11 of 118), F-22 as F-15 (7 of 67) and E-8 as KC-135 (6 of 51). Each pair is visually close from above: two Sukhoi fighters that share a similar planform, small fighters that cover only a few pixels, and two large swept-wing jets with four underwing engines.

Interestingly, the E-8 was taken for an E-3 only twice, less often than for a KC-135, so the E-3 rotodome seems to be visible enough in most scenes. The F-15 column also explains the weak AP of A13, and the F-15 row shows that this class absorbs errors from the F-16 and F-22 as well. Ultralytics builds this matrix from thresholded detections, so its counts are not the exact inputs of the AP calculation, but the pattern agrees with the per-class AP values above.

Fig. 2(b) follows validation $mAP@0.5:0.95$ over the 200 epochs. All five curves rise quickly and then flatten: every model passes 75% between epochs 19 and 25, and 80% between epoch 81 (YOLO12n) and epoch 126 (YOLOv5nu). YOLO26n starts slowest, yet it has the highest mean over the last ten epochs, 81.09%, against 80.43–80.96% for the others. The best epochs improve on the best of the first 100 epochs by only 0.70–1.17 points, so a shorter schedule would cost some accuracy, but not much. Fig. 2(c) sets test $mAP@0.5:0.95$ against training time. The three fastest models sit between 74 and 80 min with 79.77–80.04%, while YOLO26n and YOLO12n gain 0.24 and 0.41 points over YOLO11n for 19 and 31 extra minutes of training.

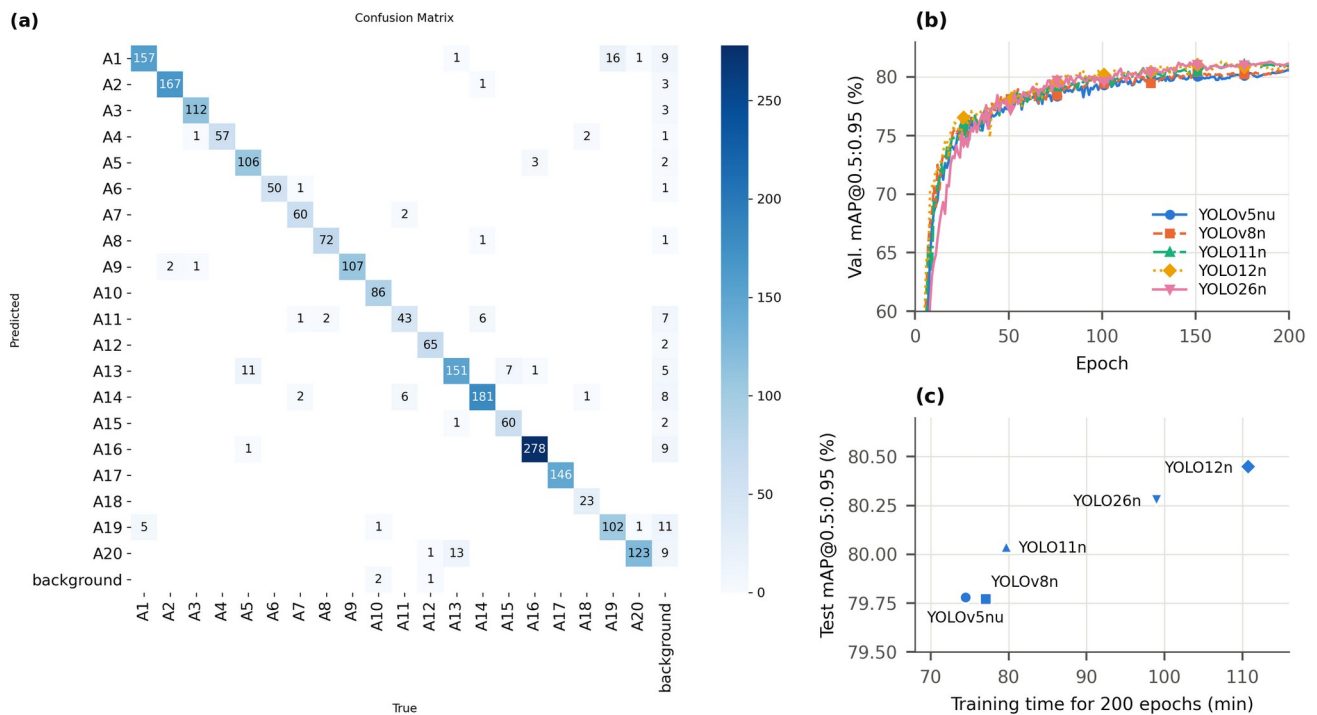


Fig. 2. (a) Test-set confusion matrix of YOLO12n in counts; columns are true classes, rows are predicted classes, and the last row and column stand for missed aircraft and background detections. (b) Validation $mAP@0.5:0.95$ of the five models during training. (c) Test $mAP@0.5:0.95$ against the recorded training time.

C. Comparison with published MAR20 results

TABLE IV. PUBLISHED MAR20 RESULTS AND THIS STUDY

Method	Box	Split	Params (M)	mAP@0.5 (%)	mAP@0.5:0.95 (%)
YOLOv5n in [2]	H	re-split ^a	1.79	94.0	–
YOLOv8n in [2]	H	re-split ^a	3.02	91.3	–
DET-YOLO [2]	H	re-split ^a	1.41	94.7	–
YOLOv8s in [3]	H	official	11.13	89.0	68.3
FGA-YOLO [3]	H	official	13.27	91.3	69.6
YOLOv8n in [4] ^b	n.r.	n.r.	–	70.2	50.0
YOLO11n full [4] ^b	n.r.	n.r.	–	75.4	54.2
YOLOv8s in [5]	H	official ^d	11.14	84.67	63.33
YOLOv8s+CBAM+WIoU [5]	H	official ^d	11.19	84.80	63.51
MFCANet [6]	O	official	–	85.96	–
PEDNet [7]	O	official	–	80.84	55.54
Shi et al. [8] ^c	O	official	–	86.71	–
LFDet [9]	O	official	16.95	87.62	–
C2IDet [10]	O	official	29.97	84.16	–
SOOD [11] ^c	O	official	38.99	85.77	–
YOLOv8n (ours)	H	7:2:1	3.02	97.21	79.77
YOLO12n (ours)	H	7:2:1	2.57	97.29	80.45

H: horizontal box; O: oriented box; n.r.: not reported; official: 1,331 training / 2,511 test images. ^a 2,765 / 308 / 769 images, 80 epochs. ^b 640 px input, nano scale, 100 epochs. ^c VOC 2007 metric. ^d 133 of the official training images held out for validation, 800 px, 150 epochs. Model names and values are as printed in each source; our split is described in Table I.

Table IV gathers the numeric MAR20 results of the studies reviewed in Section II and places our two most relevant models beside them. Taken at face value, both of our models score higher than every published figure. YOLO12n exceeds the best horizontal-box result, DET-YOLO at 94.7% [2], by 2.59 points and the best oriented result, LFDet at 87.62% [9], by 9.67 points, while using fewer parameters than every listed model except DET-YOLO and the YOLOv5n of [2]. We do not take this as evidence that a stock nano model outperforms those methods, and the reasons are worth stating.

First, the oriented detectors [6]–[11] must also regress an angle, and an oriented IoU of 0.5 is harder to reach for long, thin aircraft, so their scores sit on a different scale. Second, FGA-YOLO [3], Zhao and Tian [5] and the oriented studies train on at most 1,331 images, whereas our models saw 2,691, and part of our test subset comes from the official training list; our audit excludes only the most obvious form of overlap. Third, schedules and evaluation settings differ: DET-YOLO trained for 80 epochs and lists a confidence threshold of 0.25, while we trained for 200 epochs and evaluated at 0.001, which extends the precision–recall curve.

With these caveats in mind, the most telling comparison is between baselines. DET-YOLO’s YOLOv8n reached 91.3% mAP@0.5 on a re-split with 2,765 training images [2]; ours reached 97.21% with 2,691. Wang et al. [4] report 70.2% for the same model at the same input size. A 27-point spread for one architecture on one dataset shows how strongly the protocol drives MAR20 numbers, and it is the main reason we report the configuration, the partition and the audit alongside the scores.

Zhao and Tian [5] provide the most direct evidence on this point. Their YOLOv8s reaches 80.16% mAP@0.5:0.95 on validation images drawn from the official training list, close to our own validation and test levels, yet only 63.33% on the official test list. Our re-partitioned test subset therefore behaves much more like their validation subset than like the official test list. The official list appears to pose a harder generalization problem than a random, stratified split, and our 80.45% should not be read against their 63.51% as a 17-point improvement.

The contribution of this work is therefore not a new state of the art but a transparent reference point: five current nano detectors measured under identical, fully reported conditions and positioned against the literature. Our numbers can serve as a baseline only for studies that adopt the same partition, and we recommend that every MAR20 paper states its split, box type and metric.



Fig. 3. Ground truth (left) and YOLO12n predictions with confidence scores (right) for the first test batch. Images from [1].

D. Curves and qualitative results

In Fig. 1, mean F1 peaks at 0.95 at a confidence of 0.584 and stays close to that level from about 0.1 to 0.85, so YOLO12n is not sensitive to the exact deployment threshold. The mean precision–recall curve keeps precision above 0.97

up to a recall of about 0.8; the A11 and A13 curves are the visible exceptions.

Fig. 3 compares the ground truth with YOLO12n predictions for the first test batch. Long rows of KC-135 (A14) and F-15 (A13) aircraft, a mixed apron of C-17 (A3) and C-5

(A4) transports, and one E-8 (A11) standing beside two E-3 (A7) aircraft are all handled correctly. In a hazy image with five B-1B (A10) bombers under shelters, the model finds two, labels a third as an SU-34 (A19) and misses two; on a crowded apron, an E-3 is predicted as a KC-135 with a confidence of 0.8.

V. DISCUSSION

Five detectors spanning several years of YOLO development ended up within 0.67 points of each other on mAP@0.5:0.95, so under this protocol the architecture appears to matter less than the data split. The attention blocks of YOLO12n may help, but a single-seed design cannot separate that effect from run-to-run noise, and the model needed about 44% more training time than YOLOv8n. For a practitioner with limited compute, YOLOv5nu or YOLO11n deliver nearly the same accuracy for less training.

The remaining errors concentrate in A11, which needs features that separate near-identical airframes, and A13, which would likely benefit from a higher input resolution; in [4], raising the input from 640 to 1,024 pixels lifted YOLO11n from 63.4% to 74.4% mAP@0.5.

Two practical recommendations follow. State the split and, where possible, evaluate on the official test list, because images held out from the official training list can score about 17 points higher in mAP@0.5:0.95 than the official test images [5]. And report class-level scores and a confusion matrix alongside the mean values, because operationally important errors, such as an E-8 labeled as a tanker, disappear in the averages.

The partition is the main limitation. Our results cannot be compared directly with official-split numbers, and scene-level overlap between subsets cannot be fully excluded because the crops share airfields. We also used one seed, did not measure inference speed under a common protocol and did not evaluate on the official test list. The next step is to repeat the benchmark on the official 1,331/2,511 split with several seeds, which would place these baselines directly beside [3], [5] and [6]–[11]. Beyond the YOLO family, transformer-based competitors such as RT-DETR deserve the same treatment, and a GPU versus non-GPU comparison, where nano models are most attractive, would show what these accuracy differences cost on edge hardware.

VI. CONCLUSION

We benchmarked five nano YOLO detectors on MAR20 under a single documented configuration. On a held-out test subset of 381 images, YOLO12n obtained the highest mAP@0.5 (97.29%) and mAP@0.5:0.95 (80.45%), but all five models fell within 0.33 and 0.67 points on these measures, and YOLO12n was the slowest to train. The E-8 and F-15 were the hardest types for every model, and nearly all remaining errors were type confusions between visually similar aircraft rather than missed detections. Set against ten published studies, our scores are higher in absolute terms, yet differences in split, box type and schedule mean that they should be read as a transparent baseline for the re-partitioned protocol rather than as a new state of the art. We hope the consolidated table and the audit make future MAR20 comparisons easier to interpret.

ACKNOWLEDGMENT

This work was supported by Istiklal Science and Technology University Scientific Research Projects Coordination Unit (BAP) under Project Number 2026/2-1.

REFERENCES

- [1] W. Yu, G. Cheng, M. Wang, Y. Yao, X. Xie, X. Yao, and J. Han, "MAR20: A benchmark for military aircraft recognition in remote sensing images," *Natl. Remote Sens. Bull.*, vol. 27, no. 12, pp. 2688–2696, 2023, doi: 10.11834/jrs.20222139.
- [2] X. Chen et al., "DET-YOLO: An innovative high-performance model for detecting military aircraft in remote sensing images," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 17, pp. 17753–17771, 2024, doi: 10.1109/JSTARS.2024.3462745.
- [3] J. Wu, F. Zhao, G. Yao, and Z. Jin, "FGA-YOLO: A one-stage and high-precision detector designed for fine-grained aircraft recognition," *Neurocomputing*, vol. 618, Art. no. 129067, 2025, doi: 10.1016/j.neucom.2024.129067.
- [4] Q. Wang, R. Bai, Y. Zhang, and H. Lai, "A frequency-aware and multiscale aligned framework for fine-grained object detection in remote sensing imagery," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 19, pp. 26976–26998, 2026, doi: 10.1109/JSTARS.2026.3720469.
- [5] H. Zhao and J. Tian, "A systematic evaluation of YOLOv8s improvement strategies for remote sensing military aircraft detection," in *Proc. 7th Int. Seminar Artif. Intell., Netw. Inf. Technol. (AINIT)*, Dalian, China, 2026, pp. 842–846, doi: 10.1109/AINIT70033.2026.11557903.
- [6] H. Jiang, T. Luo, H. Peng, and G. Zhang, "MFCANet: Multiscale feature context aggregation network for oriented object detection in remote-sensing images," *IEEE Access*, vol. 12, pp. 45986–46001, 2024, doi: 10.1109/ACCESS.2024.3381539.
- [7] S. Zhu and L. Wei, "PEDNet: A proposal enhancement dynamic network for fine-grained ship detection in optical remote sensing images," *IEEE Access*, vol. 12, pp. 129813–129825, 2024, doi: 10.1109/ACCESS.2024.3457619.
- [8] T. Shi et al., "Progressive class-aware instance enhancement for aircraft detection in remote sensing imagery," *Pattern Recognit.*, vol. 164, Art. no. 111503, 2025, doi: 10.1016/j.patcog.2025.111503.
- [9] X. Xu, Z. Chen, Y. Hu, and G. Wang, "More signals matter to detection: Integrating language knowledge and frequency representations for boosting fine-grained aircraft recognition," *Neural Netw.*, vol. 187, Art. no. 107402, 2025, doi: 10.1016/j.neunet.2025.107402.
- [10] X. Xu, Z. Chen, X. Zhang, and G. Wang, "Context-aware content interaction: Grasp subtle clues for fine-grained aircraft detection," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, Art. no. 5641319, 2024, doi: 10.1109/TGRS.2024.3464851.
- [11] Y. Xi, T. Lu, X. Kang, and S. Li, "Structure-adaptive oriented object detection network for remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, Art. no. 5634013, 2024, doi: 10.1109/TGRS.2024.3432878.
- [12] J. Wang, Z. Ma, C. Wang, and N. Liu, "Few-shot object detection with contrastive enhancement loss guided by object association graph in remote sensing images," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 18, pp. 26094–26107, 2025, doi: 10.1109/JSTARS.2025.3606631.
- [13] S. Dang et al., "Edge information extraction based open-set airplane detection in remote sensing images," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 18, pp. 22094–22107, 2025, doi: 10.1109/JSTARS.2025.3600243.
- [14] G. Jocher, J. Qiu, and A. Chaurasia, "Ultralytics YOLO," ver. 8.4.14, Ultralytics, 2026, computer software. [Online]. Available: <https://github.com/ultralytics/ultralytics>